Advanced Database Models – Practical 5

NF2, eNF2 and Type Constructors – Solutions

---

1. NF2 – Car Retailer Schema

Given schema:

```
CarSales(
  Model,
  Manufacturer,
  Car(
    BuildYear,
    Kilometers,
    ListPrice,
    InterestedCustomers(
      Name,
      LastOffer,
      Telephones(Telephone)
    )
  )
)
```

Example Relation (2 tuples)

Each outer tuple = one car model. Each Car sub-tuple = one specific car (physical unit). Each InterestedCustomers sub-tuple = one customer. Telephones is a set of phone numbers.

Model	Manufacturer	Car (BuildYear, km, ListPrice)	InterestedCustomers (Name, LastOffer)	Telephones
Fiesta	Ford	Car1: 2020, 15000, 12000 Car2: 2019, 30000, 9500	Stuart, 11000 Miller, 9000	{0171-111, 0172-222} {0173-333}
Beetle	VW	Car3: 2021, 5000, 18000	Jones, 17000 Smith, 16500	{0174-444} {0175-555, 0176-666}

Fully Unnested Relation & PNF Check

Unnesting all nested relations yields one flat table where each row represents one (Model, Car, Customer, Telephone) combination:

Model	Manufacturer	BuildYear	Kilometers	ListPrice	CustName	LastOffer	Telephone
Fiesta	Ford	2020	15000	12000	Stuart	11000	0171-111
Fiesta	Ford	2020	15000	12000	Stuart	11000	0172-222
Fiesta	Ford	2020	15000	12000	Miller	9000	0173-333
Fiesta	Ford	2019	30000	9500	Stuart	11000	0171-111
...	...	...	...	...	...	...	...

PNF (Partition Normal Form) check: The unnested relation is NOT in PNF. There are massive redundancies — Manufacturer repeats for every (Car, Customer, Telephone) combination; ListPrice repeats for every customer/phone combination of the same car. PNF would require lossless decomposition so that each fact appears exactly once.

Equivalent Relational Schema

```
CarModel(Model PK, Manufacturer)

Car(CarID PK, Model FK->CarModel, BuildYear, Kilometers, ListPrice)

Customer(CustomerID PK, Name)

Interest(CarID FK->Car, CustomerID FK->Customer, LastOffer
         PK: (CarID, CustomerID))

Telephone(CustomerID FK->Customer, Telephone
          PK: (CustomerID, Telephone))
```

Queries in Extended Relational Algebra and SQL:2003

Q1 – List prices of Citroens

Extended Relational Algebra (NF2 unnesting operator μ):

```
pi_{ListPrice} (sigma_{Manufacturer='Citroen'} (
    mu_{Car} (CarSales)         -- unnest Car
))
```

SQL:2003 (using UNNEST):

```
SELECT c.ListPrice
FROM   CarSales cs,
       UNNEST(cs.Car) AS c
WHERE  cs.Manufacturer = 'Citroen';
```

Q2 – Offers for a VW Beetle

Extended Relational Algebra:

```
pi_{LastOffer} (sigma_{Model='Beetle' AND Manufacturer='VW'} (
    mu_{Car} (mu_{InterestedCustomers} (CarSales))
))
```

SQL:2003:

```
SELECT ic.LastOffer
FROM   CarSales cs,
       UNNEST(cs.Car) AS c,
       UNNEST(c.InterestedCustomers) AS ic
WHERE  cs.Model = 'Beetle'
  AND  cs.Manufacturer = 'VW';
```

Q3 – Phone numbers of Mr. Stuart interested in Ford Fiesta

Extended Relational Algebra:

```
pi_{Telephone} (sigma_{Model='Fiesta' AND Manufacturer='Ford' AND Name='Stuart'} (
    mu_{Telephones} (mu_{InterestedCustomers} (mu_{Car} (CarSales)))
))
```

SQL:2003:

```
SELECT t.Telephone
FROM   CarSales cs,
       UNNEST(cs.Car) AS c,
       UNNEST(c.InterestedCustomers) AS ic,
       UNNEST(ic.Telephones) AS t
WHERE  cs.Model       = 'Fiesta'
  AND  cs.Manufacturer = 'Ford'
  AND  ic.Name         = 'Stuart';
```

---

2. eNF2 and Type Constructors – Online Auction

Entities: Member (name, email), Auction (title, description, startprice), Bid (price) by a member for an auction, Rating (date, score, comment) of a member by another member, Image (up to 3 per auction).

Relational Schema (1NF)

```
Member  (MemberID PK, Name, Email)

Auction (AuctionID PK, Title, Description, StartPrice,
         SellerID FK->Member)

Bid     (BidID PK, AuctionID FK->Auction,
         BidderID FK->Member, Price)

Rating  (RatingID PK, RatedMemberID FK->Member,
         RaterMemberID FK->Member, Date, Score, Comment)

-- Images: max 3 per auction -> store as 3 nullable columns (simple)
-- OR: separate table (flexible, recommended)
AuctionImage (AuctionID FK->Auction, ImageNo CHECK(1..3), ImageData
              PK: (AuctionID, ImageNo))
```

eNF2 Schema with Type Constructors

eNF2 allows set-valued and tuple-valued attributes using LIST, SET, and TUPLE constructors. This eliminates join tables and models the domain more naturally.

```
Member(
  MemberID,
  Name,
  Email,
  Ratings: LIST<TUPLE<Date, Score: INT, Comment, RaterID>>
)

Auction(
  AuctionID,
  Title,
  Description,
  StartPrice,
  SellerID  -> Member,
  Bids:   LIST<TUPLE<BidderID -> Member, Price>>  -- ordered by time
  Images: LIST<BLOB>[0..3]                        -- max 3 images
)
```

Design rationale: Bids are nested inside Auction because a bid only makes sense in the context of an auction. Ratings are nested in Member because they describe the member. Images are a bounded list (max 3) inside Auction. Cross-references (SellerID, BidderID, RaterID) remain as foreign keys since they reference the independent Member entity.

Implementation in SQL:2003

SQL:2003 supports structured user-defined types (UDTs), ROW types, and arrays.

```
-- Define row types
CREATE TYPE BidType AS (
    BidderID  INT,
    Price     DECIMAL(10,2)
) NOT FINAL;

CREATE TYPE RatingType AS (
    RaterID   INT,
```

```
    RateDate  DATE,
    Score     INT,
    Comment   VARCHAR(500)
) NOT FINAL;

-- Member table with nested ratings array
CREATE TABLE Member (
    MemberID  INT PRIMARY KEY,
    Name      VARCHAR(100),
    Email     VARCHAR(100),
    Ratings   RatingType ARRAY DEFAULT ARRAY[]
);

-- Auction table with nested bids and images
CREATE TABLE Auction (
    AuctionID   INT PRIMARY KEY,
    Title       VARCHAR(200),
    Description CLOB,
    StartPrice  DECIMAL(10,2),
    SellerID    INT REFERENCES Member(MemberID),
    Bids        BidType ARRAY DEFAULT ARRAY[],
    Images      BLOB ARRAY[3]    -- max 3 images
);
```

Implementation in Oracle 10g

Oracle uses OBJECT types and nested TABLE / VARRAY collections.

```
-- Object types
CREATE OR REPLACE TYPE BidObj AS OBJECT (
    BidderID  NUMBER,
    Price     NUMBER(10,2)
);
/
CREATE OR REPLACE TYPE BidList AS TABLE OF BidObj;
/

CREATE OR REPLACE TYPE RatingObj AS OBJECT (
    RaterID  NUMBER,
    RateDate DATE,
    Score    NUMBER(1),
    Comment  VARCHAR2(500)
);
/
CREATE OR REPLACE TYPE RatingList AS TABLE OF RatingObj;
/

-- Images: max 3 -> use VARRAY
CREATE OR REPLACE TYPE ImageArray AS VARRAY(3) OF BLOB;
/

-- Tables
CREATE TABLE Member (
    MemberID  NUMBER PRIMARY KEY,
    Name      VARCHAR2(100),
    Email     VARCHAR2(100),
    Ratings   RatingList
) NESTED TABLE Ratings STORE AS MemberRatings_Tab;

CREATE TABLE Auction (
```

```
    AuctionID   NUMBER PRIMARY KEY,
    Title       VARCHAR2(200),
    Description CLOB,
    StartPrice  NUMBER(10,2),
    SellerID    NUMBER REFERENCES Member(MemberID),
    Bids        BidList,
    Images      ImageArray
) NESTED TABLE Bids STORE AS AuctionBids_Tab;
```

Key difference SQL:2003 vs Oracle 10g: SQL:2003 uses ARRAY with a fixed declared element type. Oracle uses VARRAY (fixed-max ordered collection) or nested TABLE (unordered, unbounded) with separately stored backing tables. The NESTED TABLE...STORE AS clause tells Oracle where to physically store the nested rows.